Resume de Python

	General
Hello world	`print ("Hello world!")`
Comentarios	`#to end of line`

Comentarios	`"""`
	`This is multi-line`
	`comment`
	`"""`

Librerías	`import sys` `import os`

Lineas largas	`x = 1+2+3+ \`
	`    4+5`
	`x = (1+2+3+`
	`     4+5)`

String largo	`s = "Esto es un texto "+ \` `    "muy largo"`

Print	`print("Hola")`
	`print("Vel %02d %7.2f km/h" % (i,x))`
	`print("Ver {0:02d} {1:7.2f} km/h".format(i,x))`
	`print("Valor:",end=" ") #termina con " " en lugar de \n`

	Condiciones
Condiciones	`if a>1000:` `    print(a,"is big")`

Condiciones	`if a>1000:`
	`      print (a,"is big")`
	`elif a>100:`
	`      print (a,"is medium")`
	`else:`
	`      print (a,"is small")`

Operadores R	`==  !=  >  <  >=  <=`
Operadores L	`and  or  not`
	Operadores matemáticos
	`+  -  *  %`
	`/  --> division real   // --> floor`
	`**  --> potencia`
	`+=  -=  *=`
	Strings
Inicializar	`a = "Hello, World!"`
Leer de teclado	`a = input()`
Un caracter	`print(a[1])   # --> e`
Rango	`print(a[2:5]) # --> llo`
Hasta el final	`print(a[2:])  # --> llo, World!`
Desde el final	`print(a[-6:-2])  # --> Worl`
Longitud	`lo=len(a)`
Copiar	`b = a`
Comparar	`if b==a:` `    print("son iguales")`

Funciones	`pos=a.find("World")`
	`up=a.upper()`
	`lo=a.lower()`
	`lista=a.split(" ")`
	`st=" ".join(lista) #junta lista en string`
	`zz=a.replace(";",".") #cambia ; por .`

	Conversiones
string <-> int	`a = int(string),  string=str(num)`
float -> int	`a = int(x)`

num -> string (con formato)	`st = str(num) #conversión sin control de formato`
	`st = "%03d" % num  #conversión con formato`
	`print("%03d" % num)`
	`st = "%02d/%02d/%04d" % (dia,mes,aNo)`

	Listas, Tuplas y Diccionarios (~estructuras)
Lista	`ver = [ 1, 2, 3, 5, 7, 11, 13]`
Lista	`colors = ['red', 'blue', 'green']`
Lista	`lista = ["x", 11.8, 7]  #Lista, mutable`
Tupla	`tupla = ("x", 11.8, 7)  #Tupla, inmutable`

Funciones inplace	`lista.append(elemento)`
	`lista.extend(lista2)`
	`lista.sort()`
	`lista.remove(elemento)`
	`lista.insert(pos,elemento)`
	`lista.reverse()`

Funciones	`lista2 = lista1[:]  #copiar una lista`
	`lista = lista1+lista2`
	`tf = "red" in lista  #True or False`
	`num = lista.count("red")`
	`pos=lista.index(elemento)`
	`lon = len(lista)`
	`s=sum(lista); M=max(lista); m=min(lista)`

Diccionario	`car = { "brand": "Ford", "model": "Mustang", "year": 1964 }`
	`#inicialización alternativa:`
	`car = dict(brand="Ford", model="Mustang", year=1964)`
	`car["km"]=78345  #Añade nuevo campo "km"`

	`print(car["year"])` `if "year" in car.keys(): print("Existe campo year")`
	`car2 = car.copy()`
	Bucles

For numerico	`for i in range(10): #bucle del 0 al 9`
	`  print(i)`
	`#Ejemplo: 5 números aleatorios de 0 a 9`
	`import random`
	`r=[]`
	`for i in range(5):`
	`  r.append(random.randint(0,9))# 0 y 9 son válidos`
	`print(r)`

For en listas 🙂	`colors = ['red', 'blue', 'green']`
	`for c in colors:`
	`  print(c)`

For en listas 😕	`for i in range(len(colors))` `  print(colors[i])`

For en dict	`for x in car.keys():`
	`  print(x)  #keys`
	`  print(car[x]) #values`

For en dict	`for x in car.values():` `  print(x)  #values`
For en dict	`for k,v in car.items():` `  print(k, v)  #key and value`

For 2 variables a la vez	`impar=[1,3,5,7]`
	`par=[2,4,6,8]`
	`for im,pa in zip(impar,par):`
	`  print(im,pa)`

while	`a=int(input("Numero entero positivo: "))`
	`while a<0:`
	`  print("Error: debe ser positivo")`
	`  a=int(input("Numero entero positivo: "))`

	Funciones

Definición	`def Cuadrado(x):`
	`  res=x*x`
	`  return res  #devuelve un resultado`

Varios resultados	`def Cambiar(a,b):`
	`  return b,a  #devuelve dos resultados`
	`#Programa principal:`
	`x=7; y=8`
	`print("x vale ",str(x),"y vale ",str(y))`
	`x,y=Cambiar(x,y)`
	`print("Ahora x vale ",str(x),"mientras que y vale",str(y))`
	`----------`
	`x vale  7 y vale  8`
	`Ahora x vale  8 mientras que y vale 7`

	Archivos de texto

Abrir / Cerrar	`f=open("prueba.txt","r")  #r =read, w =write, a =append`
	`linea=f.readline()  #r+ =read then write, w+ =write then read`
	`f.close()`

Abrir / Cerrar	`with open("prueba.txt","r") as f:`
	`  linea=f.readline()`
	
	`#Con este método no hace falta close.`

Abrir / Cerrar control errores	`try:`
	`  with open("prueba.txt", "r") as f:`
	`  linea=f.readline()`
	`  #más lectura aquí`
	`except IOError:`
	`  print("Oops, error al abrir")`

Funciones de Lectura	`#leer una linea en un string`
	`linea=f.readline()`
	
	`#lee todas las líneas en una lista de strings`
	`lineas=f.readlines()`
	
	`#lee todo el archivo en un único string`
	`todo=f.read()`
	
	`#lee todo el archivo línea a línea`
	`line=file.readline()`
	`while(line):`
	`  #aquí trabajo con la línea`
	`  print(line)`
	`  line=file.readline()`
	`f.close()`

Funciones de Escritura	`f = open("prueba.txt", "w")`
	`f.write("Creo o borro el fichero.")`
	`f.close()`
	
	`f = open("prueba.txt", "a")`
	`f.write("Añado este texto al final")`
	`f.close()`

	Archivos Binarios
Escritura	`with open("binario.dat","wb") as fb:` `  pickle.dump(var,fb)  #var es una variable de cualquier tipo`
Lectura	`with open("binario.dat","rb") as fb:` `  var=pickle.load(fb)  #var es una variable de cualquier tipo`

	Pandas

Leer CSV, TSV	`import pandas as pd`
	`import numpy as np`
	`datos=pd.read_csv("fichero.csv",sep="\t") #separadores típicos: '\t' ',' ';'`
	`#Aspecto de los datos (ver nombres de columnas):`
	`datos.head()`
	`datos.info()`
	`datos.describe()`
	`#para escribir dataframe a CSV:`
	`datos.to_csv('resultados.csv')`

Read/Write Excel	`datos=pd.read_excel('North_Dakota.xlsx', 'tabla1')` `datos.to_excel('resultados.xlsx')`

Acceso a datos	`#Datos de la columna Latitude:`
	`lat=datos["Latitude"]`
	`lista_lat=datos["Latitude"].tolist() #Convertido a lista normal`
	`lat7=datos["Latitude"].values[7] #Equivale a lat7=lista_lat[7] pero sin crear la lista`
	`datos["Latitude"].values[7]=40.43 #modificar valor`
	`#Columna Latitude filtrada por el valor de la columna tipo_vivienda:`
	`lat_hh=datos[datos["tipo_vivienda"]=="household"]["Latitude"]`
	`datos['precio']=0  #Añado columna de precio con valores 0`
	`datos['precio']=lista #Añado columna de precio con valores de una lista`

	Webscraping / JSON

Obtener html	`import requests`
	`resultado=requests.get("https://twitter.com")`
	`if resultado.status_code != 200: print("Error de URL")`
	`mi_html=resultado.content`

BeautifulSoup	`from bs4 import BeautifulSoup`
	`soup=BeautifulSoup(mi_html)`
	`anchors=soup.find_all("a") #anchors es una lista bs4 con todas las etiquetas <a>`
	`anchors[0]['href'] #campo href de la primera etiqueta <a>`
	`anchors[0].string  #texto entre <a...> y </a>`
	`#forma compacta de obtener todos los links de la página:`
	`links = [a['href'] for a in anchors if a.has_key('href')]`

JSON	`dicc=resultado.json()  #convierte a diccionario el resultado de requests`
	`#Un JSON puede ser una combinación de listas y diccionarios`
	`#Hay que analizar la estructura en cada caso`

	Gráficas con matplotlib

plot(x,y)	`import matplotlib.pyplot as plt`
	`import numpy as np`
	`x=np.arange(0,np.pi,0.1)   #esto genera [0, 0.1, 0.2, ... 3.1]`
	`x=np.linspace(0,np.pi,100) #esto genera 100 muestras. x[0] es 0, x[-1] es PI`
	`y=np.sin(2*np.pi*x)`
	`plt.plot(x,y)`

Estilos de plot	`plt.plot(x,y,'r-') #linea roja`
	`plt.plot(x,y,'ro') #puntos rojos`
	`plt.plot(x,y,'ro-') #puntos y lineas rojas`

Subplot	`plt.subplot(rows,cols,num) #Ej. subplot(3,2,1)`

Textos	`plt.title("Titulo grafica")`
	`plt.xlabel("texto eje x")`
	`plt.ylabel("texto eje y")`
	`plt.text(x0,y0,"Texto")`
	`dias=['L','M','X','J','V','S','D'] #Textos`
	`xdias=[0,1,2,3,4,5,6] #valores eje X`
	`plt.xticks(xdias,dias)`

Tipos de gráficos	`plt.figure(figsize=(6,4)). #width=6, height=4. Util para crear subplots`
	`plt.plot(x,y)`
	`plt.scatter(x,y)`
	`plt.bar(names,values)`
	`plt.hist(x,50)`
	`names = ['Frogs', 'Hogs', 'Dogs', 'Logs']`
	`values = [120, 80, 40, 100]`
	`plt.pie(values,labels=names,autopct='%5.1f%%') #muestra porcentajes en formato %5.1f`

Funciones auxiliares	`plt.axis([xmin,xmax,ymin,ymax])`
	`plt.grid(True)`
	`plt.yscale('linear') #'log'`
	`plt.legend`
	`plt.show()`